Transformer Parameter Correction

Problem Premise

The MG-RAVENS format stores detailed transformer data in the `PowerTransformer` objects. During data acquisition, conversion, or manual editing, the following faults are commonly introduced:

- **Deletion errors** – the star-impedance (`TransformerStarImpedance.r/x`) or core-admittance (`TransformerCoreAdmittance.g/b`) fields are set to zero due to the user not specifying them.
- **Multiplicative / additive noise** – physical parameters are incorrectly specified by the user often by orders of magnitude due to unit conversion errors.

When such errors are present, a power-flow solver (PowerModelsDistribution) often returns an infeasible solution or one very far off from true operating parameters. The goal of this work is to **recover the original transformer parameters** for each grid given a corrupted MG-RAVENS file.

Formulation

Formal Problem Definition

For a single grid let

- $T = {t_1,\dots,t_{N_T}}$ be the set of transformer ends.
- Each end t is described by a vector of physical parameters $\mathbf{p}_t = (r_t, x_t, g_t, b_t)$.
- The corrupted dataset provides $\tilde{\mathbf{p}}_t$ where some entries are zero or perturbed by a random affine transformation.

Define a regression function f_θ (parameterised by a neural network) that maps the graph representation $\mathcal{G}$ to a predicted parameter set $\hat{\mathbf{p}} = f_\theta(\mathcal{G})$.

The optimisation problem is

$\min_{\theta}; \mathcal{L}\bigl(\hat{\mathbf{p}}, \mathbf{p}^{\text{clean}}\bigr) + \lambda,\mathcal{C}\bigl(\hat{\mathbf{p}}\bigr)$

where

- $\mathcal{L}$ is a weighted mean-squared error (see **Loss Functions**).
- $\mathcal{C}$ is an *infeasibility penalty* that evaluates a power-flow simulation on the predicted grid; it returns a scalar that grows with the total violation of transformer constraints.
- λ balances the two terms.

Formal Data Description

Symbol	Meaning

Symbol	Meaning
`raw_mgr`	List of triples `[file_name, ravens_data, original_data]`.
`PowerTransformer`	Dictionary of transformer objects; each contains a list
`PowerTransformer.PowerTransformerEnd`.	
`TransformerEnd.StarImpedance`	Contains `TransformerStarImpedance.r` and
`TransformerStarImpedance.x`.	
`TransformerEnd.CoreAdmittance`	Contains `TransformerCoreAdmittance.g` and
`TransformerCoreAdmittance.b`.	
`generate_trans_error`	Synthetic data generator that injects the three error
types above. Returns `(corrupted_mgr, clean_mgr)`.	
`MGTransformerDataset`	PyTorch-Geometric `InMemoryDataset` that yields a pair
`(corrupted, clean)` where the clean graph is stored in the `y` field.	
`y` (target)	Holds `x`, `edge_attr`, `edge_index`, `max_phase`,
`file_name`, and the path to the pristine JSON file (`raw_mgr`).	
`edge_features` (ML input)	For each transformer end a dictionary with keys

`R`, `X`, `G`, `B`, `phases`, `Edge Type`, `Tap`, `Shift`. Matrices are padded to a square of size `max_phase`. | | `prediction` (model output) | Tensor of shape `(num_edges, 40)`; the 40 entries follow the ordering used in `dict_to_pyg`. |

Methods

Deterministic Iterative Optimizer

`Trans_Iterative_Optimizer` implements a rule-based, gradient-free search that re-adjusts transformer parameters directly in the MG-RAVENS structure.

1. **Initial feasibility check** – run a power-flow (`_run_pf`). If the grid is already feasible, the original parameters are returned unchanged.
2. **Infeasibility analysis** – `analyze_branch_infeasibility` extracts four violation metrics (`r_infeasibility`, `x_infeasibility`, `g_infeasibility`, `b_infeasibility`) for every

branch/transformer from the PowerModelsDistribution solution.
3. **Selection of candidates** – the most infeasible transformers (top 40 % or at least one) are selected for modification.
4. **Parameter-specific updates** – `apply_parameter_changes` scales the offending parameter(s) by a factor that depends on the measured infeasibility and a global learning rate (`self.learning_rate`). The scaling respects physical bounds (non-negative values) and differs for resistance/reactance (down-scale) versus magnetising susceptance (up-scale).
5. **Score evaluation** – after each modification the grid is re-solved; the total infeasibility score is summed across all transformers.
6. **Greedy acceptance** – any modification that strictly reduces the score is kept; otherwise the learning rate is reduced (multiplied by `0.8`).
7. **Termination** – either the grid becomes feasible or the maximum number of iterations (`max_iter`) is reached; the best configuration found throughout the run is returned.

The optimizer works directly on the **ML data dictionary (`edge_features`)**, so it can be used as a post-processing step after a neural network prediction or as a baseline for comparison.

Graph-Neural-Network Models

Two GNN architectures predict the full set of transformer parameters simultaneously.

SimpleGNN (regression head)

- **Message passing** – `transport_distance` layers of PNA convolution (`PNAConv`) with aggregators `mean, min, max, std` and scalers `identity, amplification, attenuation`.
- **Node normalisation** – batch normalisation after each convolution.
- **Edge representation** – for each edge the source node embedding, target node embedding and original edge attributes are concatenated.
- **Deep MLP head** – 10 linear layers with ReLU, dropout (progressively decreasing drop-out rates) ending in an output of size **40**, matching the flattened transformer feature vector described in the dataset section.
- **Output** – a tensor `(num_edges, 40)` that is interpreted directly as the predicted parameter matrix (no sigmoid because regression values can be negative before clamping).

AttnGNN (edge-attention augmentation)

- **Edge-attention module** – projects the concatenated source/target/edge features to a hidden dimension, applies multi-head self-attention, and then passes through layer-norm and a feed-forward network.
- The attention output replaces the original edge attributes before the PNA convolutions, allowing the network to capture higher-order interactions between transformer ends.
- The rest of the pipeline (PNA layers, MLP head) is identical to `SimpleGNN`.

Both models are compatible with the `MGTransformerDataset` loader; during training the clean graph is accessed through the `y` field.

Loss Functions

- **WeightedMSELoss**
  - Computes a weighted mean-squared error.
  - Diagonal resistance/reactance entries receive a higher weight (3); off-diagonal entries receive a lower weight (1).
  - An optional small penalty on negative predictions can be added via `penalty_strength`.
- **PI_WMSE_Loss**
  - Extends **WeightedMSELoss** by adding a physics-informed penalty term.
  - With probability `test_percentage`, a neural network predicts the amount of system demand that is not met after running a power-flow simulation (`run_pf` → `system_demand_not_met`).
  - The predicted infeasibility is multiplied by `inf_penalty` and added to the loss, producing a differentiable, physics-aware component.

Both losses expect the full `Data` object (`target_grid`) so they can fetch the clean edge attributes from `target_grid.y["edge_attr"]`. They raise a clear exception if shapes do not match.

Data Pipeline (MGTransformerDataset)

1. **Load clean MG-RAVENS files** using `MGRavensDataset`.
2. **Inject errors** via `generate_trans_error`, producing a corrupted manager (`corrupted_mgr`) and a clean copy (`clean_copies`).
3. **Convert to PyG** – `dict_to_pyg` builds node features, edge index, and a uniform edge-feature matrix padded to `max_phase`. The clean graph is stored in the `y` attribute of the corrupted `Data` object.
4. **Split** – deterministic 70 % / 15 % / 15 % split (train/val/test) using a fixed random seed.
5. **Return** – `__getitem__` yields a single `Data` object containing the corrupted input (`x`, `edge_index`, `edge_attr`) and the clean target in `y`.

During inference the helper `update_mgr` reconstructs a full MG-RAVENS JSON file from the predicted tensor:

- The tensor is turned into plain Python objects (`_to_python`).
- Each transformer end receives the diagonal entries of the predicted `R`, `X`, `G`, `B` matrices.
- The resulting JSON can be passed to `run_pf` for feasibility testing.

Results

Dataset: 20,000 20-node subsets of IEEE8500
Training: 100 epochs
Metric: Validation MSE

Model Architecture	Validation MSE
Fully Connected MLP Head	672.21
7× PNA Convolutions w/ MLP Head	114.80

Model Architecture	Validation MSE
14× PNA Convolutions w/ MLP Head	91.98
18× PNA Convolutions w/ MLP Head	94.30
20× PNA Convolutions w/ MLP Head	100.84
1x Edge Attention w/ MLP Head	[VALUE]
2x Edge Attention w/ MLP Head	[VALUE]
3x Edge Attention w/ MLP Head	[VALUE]
4x Edge Attention w/ MLP Head	[VALUE]